

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES

Béni Mellal

RAPPORT DE TRAVAUX PRATIQUES

Docker – Application Multi-Conteneurs

Module	Génie Logiciel
Encadrant	Pr. BE. ELBAGHAZAOUI
Réalisé par	Fatima Ezzahra IGHIL
Filière	Transformation Digitale Industrielle (TDI)
Niveau	2ème année cycle ingénieur
Date	26 avril 2026

1. Introduction

Ce TP a pour objectif de mettre en pratique les concepts fondamentaux de Docker et Docker Compose à travers la création et l'orchestration d'une application web multi-conteneurs. L'application déployée repose sur trois services principaux :

- Un serveur web Nginx servant le frontend (page HTML statique)
- Une base de données PostgreSQL pour le stockage des données
- L'outil pgAdmin 4 pour la gestion graphique de la base de données

2. Structure du Projet

Le projet est organisé selon l'arborescence suivante :

```
project/
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── .dockerignore
├── backend/
│   └── db.env
└── docker-compose.yml
```

Les captures ci-dessous montrent l'arborescence réelle du projet depuis l'explorateur de fichiers Windows :

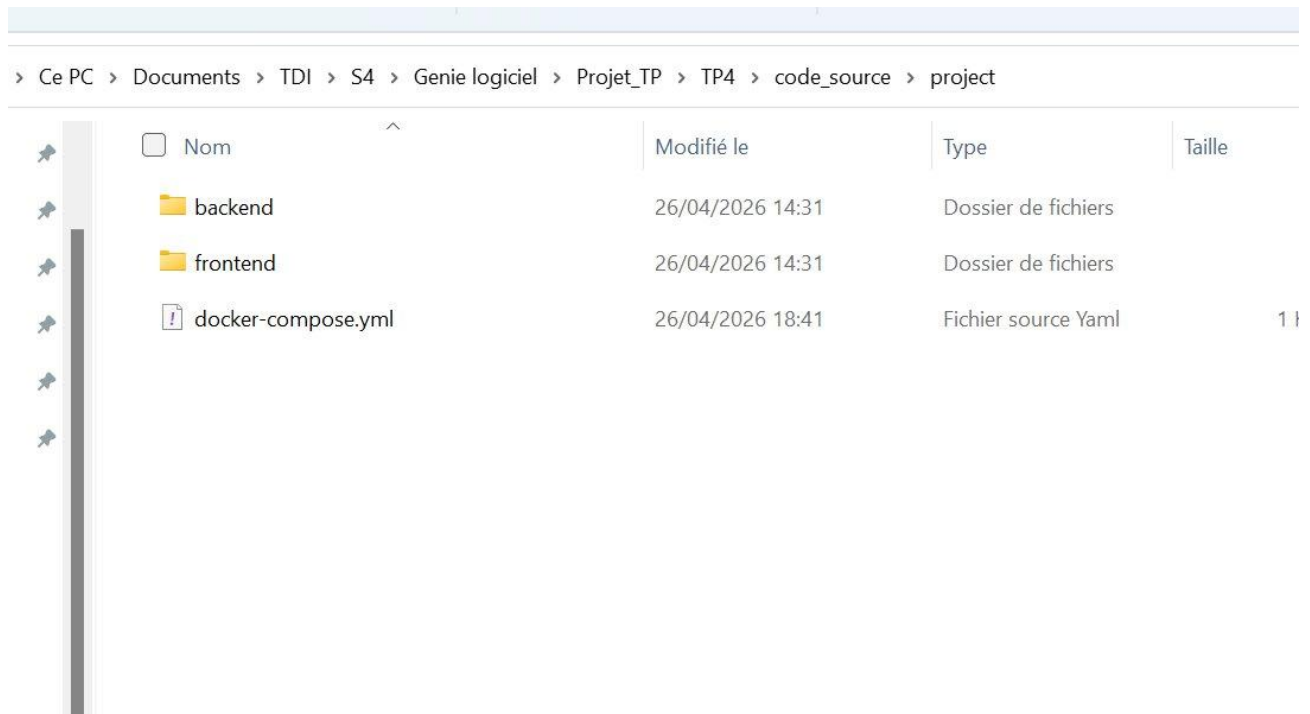


Figure 1 – Dossier racine du projet (frontend/, backend/, docker-compose.yml)

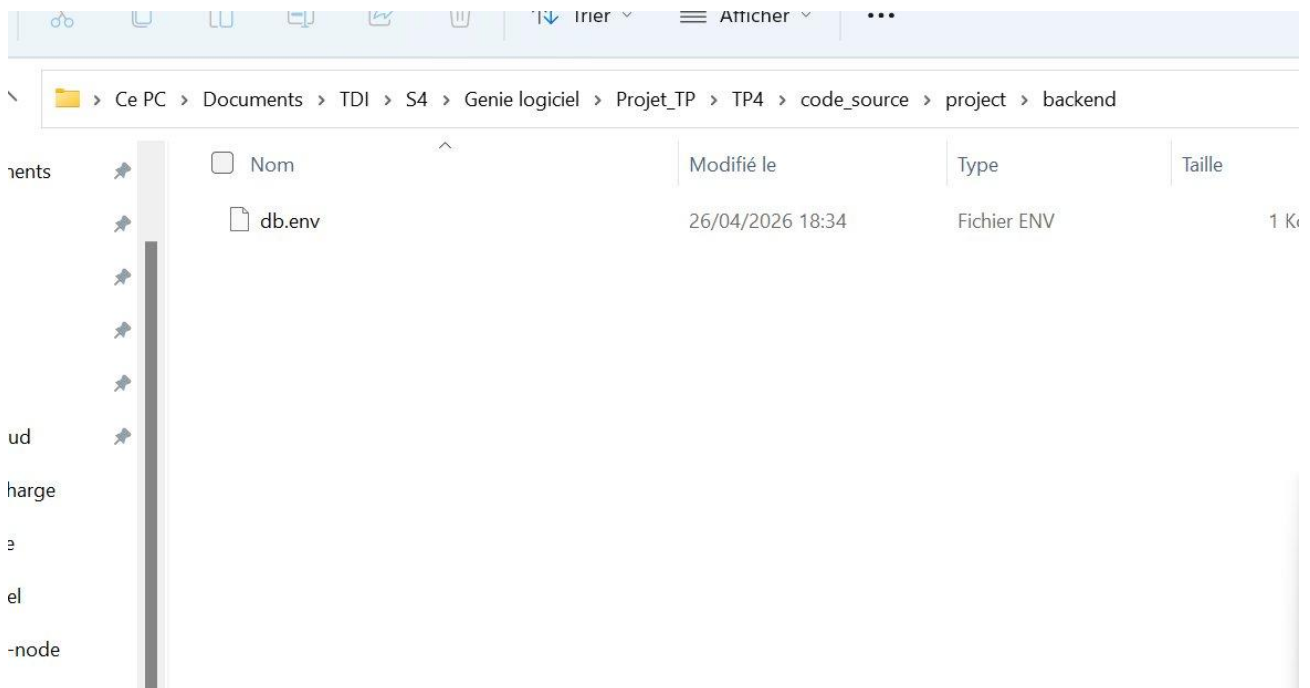


Figure 2 – Contenu du dossier backend/ (fichier db.env)

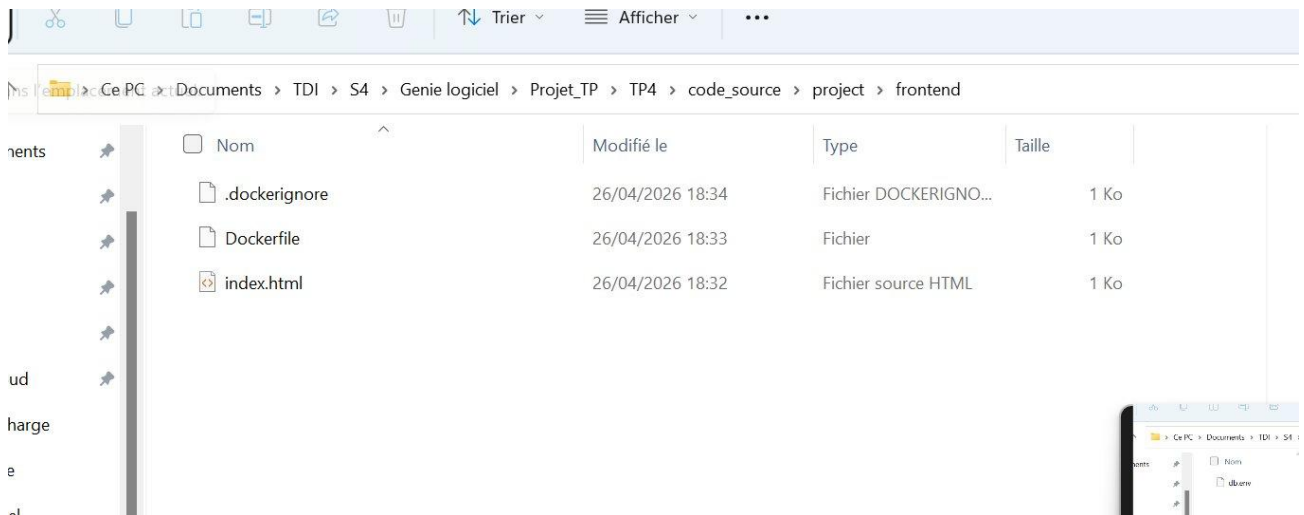


Figure 3 – Contenu du dossier frontend/ (Dockerfile, index.html, .dockerignore)

3. Fichiers de Configuration

3.1 frontend/index.html

Une page HTML simple affichant le message de bienvenue de l'application dockerisée :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Dockerized App</title>
  </head>
  <body>
    <h1>Bienvenue dans l'application Dockerisée</h1>
  </body>
</html>
```

3.2 frontend/Dockerfile

L'image Docker du frontend est basée sur nginx:alpine. Le fichier HTML est copié dans le répertoire servi par Nginx et le port 80 est exposé :

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

3.3 backend/db.env

Ce fichier centralise les variables d'environnement nécessaires à l'initialisation de PostgreSQL :

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
```

3.4 docker-compose.yml

Le fichier docker-compose.yml orchestre les trois services connectés via un réseau virtuel app-network :

- Service frontend : construit depuis le Dockerfile local, exposé sur le port 8081 de l'hôte.
- Service db : image officielle postgres:13, variables chargées depuis db.env, volume persistant db-data.
- Service pgadmin : image dpage/pgadmin4, accessible sur le port 5050, dépend du service db.

4. Déploiement et Tests

4.1 Construction et démarrage des conteneurs

La commande suivante construit les images et démarre l'ensemble des services :

```
docker-compose up --build
```

Les logs ci-dessous confirment le démarrage correct de chaque conteneur : Nginx, PostgreSQL 13 et pgAdmin 4 sont tous opérationnels.

```

C:\Windows\system32\cmd.exe - docker-compose up --build
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: start worker processes
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: start worker process 30
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: start worker process 31
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: start worker process 32
frontend-1 | 2026/04/26 17:42:11 [notice] 1#1: start worker process 33

db-1 | 2026-04-26 17:42:11.581 UTC [1] LOG: starting PostgreSQL 13.23 (Debian 13.23-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1 | 2026-04-26 17:42:11.593 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-04-26 17:42:11.595 UTC [1] LOG: listening on IPv6 address "::", port 5432
db-1 | 2026-04-26 17:42:11.625 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-04-26 17:42:11.662 UTC [27] LOG: database system was shut down at 2026-04-26 17:41:59 UTC
db-1 | 2026-04-26 17:42:11.729 UTC [1] LOG: database system is ready to accept connections
pgadmin-1 | email config is {'CHECK_EMAIL_DELIVERABILITY': False, 'ALLOW_SPECIAL_EMAIL_DOMAINS': [], 'GLOBALLY_DELIVERABLE': True}
pgadmin-1 | /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1 | return (ssh_host,
pgadmin-1 | NOTE: Configuring authentication for SERVER mode.
pgadmin-1 |
pgadmin-1 | pgAdmin 4 - Application Initialisation
pgadmin-1 | =====
pgadmin-1 | postfix/postlog: starting the Postfix mail system
pgadmin-1 | [2026-04-26 17:43:00 +0000] [1] [INFO] Starting gunicorn 23.0.0
pgadmin-1 | [2026-04-26 17:43:00 +0000] [1] [INFO] Listening at: http://[::]:80 (1)
pgadmin-1 | [2026-04-26 17:43:00 +0000] [1] [INFO] Using worker: gthread
pgadmin-1 | [2026-04-26 17:43:00 +0000] [123] [INFO] Booting worker with pid: 123
pgadmin-1 | /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1 | return (ssh_host,
frontend-1 | 172.29.0.1 - - [26/Apr/2026:17:44:30 +0000] "GET / HTTP/1.1" 200 273 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
frontend-1 | 2026/04/26 17:44:30 [error] 32#32: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.29.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "localhost:8081", referrer: "http://localhost:8081/"
frontend-1 | 172.29.0.1 - - [26/Apr/2026:17:44:30 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8081/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"

```

Figure 4 – Logs de démarrage : docker-compose up --build (Nginx, PostgreSQL, pgAdmin)

4.2 Vérification du frontend

Le frontend est accessible à l'adresse <http://localhost:8081>. La page affiche correctement le message attendu, confirmant que Nginx sert bien le fichier index.html depuis le conteneur.



Figure 5 – Page frontend accessible sur <http://localhost:8081>

4.3 Connexion à pgAdmin et tableau de bord

pgAdmin 4 est accessible à l'adresse <http://localhost:5050> avec les identifiants `admin@admin.com` / `admin`. Après ajout du serveur PostgreSQL (host: `db`, port: `5432`), le tableau de bord affiche les métriques en temps réel.

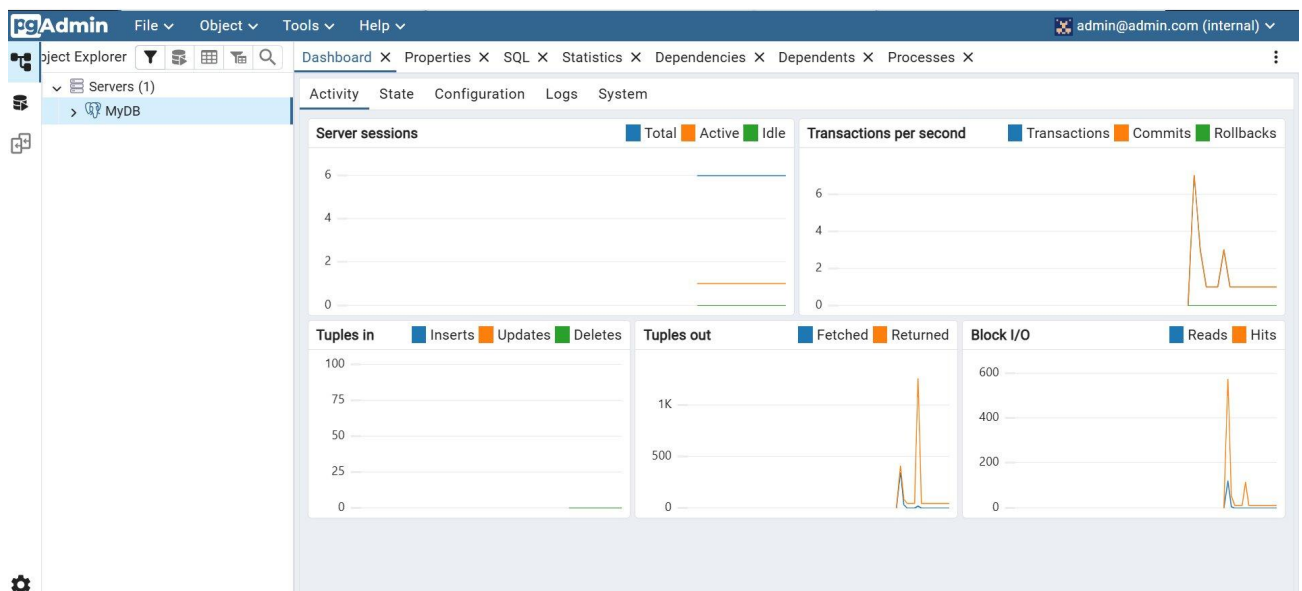


Figure 6 – Tableau de bord pgAdmin avec le serveur MyDB connecté

4.4 Création de la table et insertion de données

Via l'éditeur SQL de pgAdmin, une table `users` a été créée dans la base `myapp` avec les colonnes `id` (clé primaire), `name` et `email`. Deux enregistrements ont ensuite été insérés :

```

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
INSERT INTO users (name, email) VALUES
    ('Fatima', 'fatima@email.com'),
    ('Ali', 'ali@email.com');
SELECT * FROM users;

```

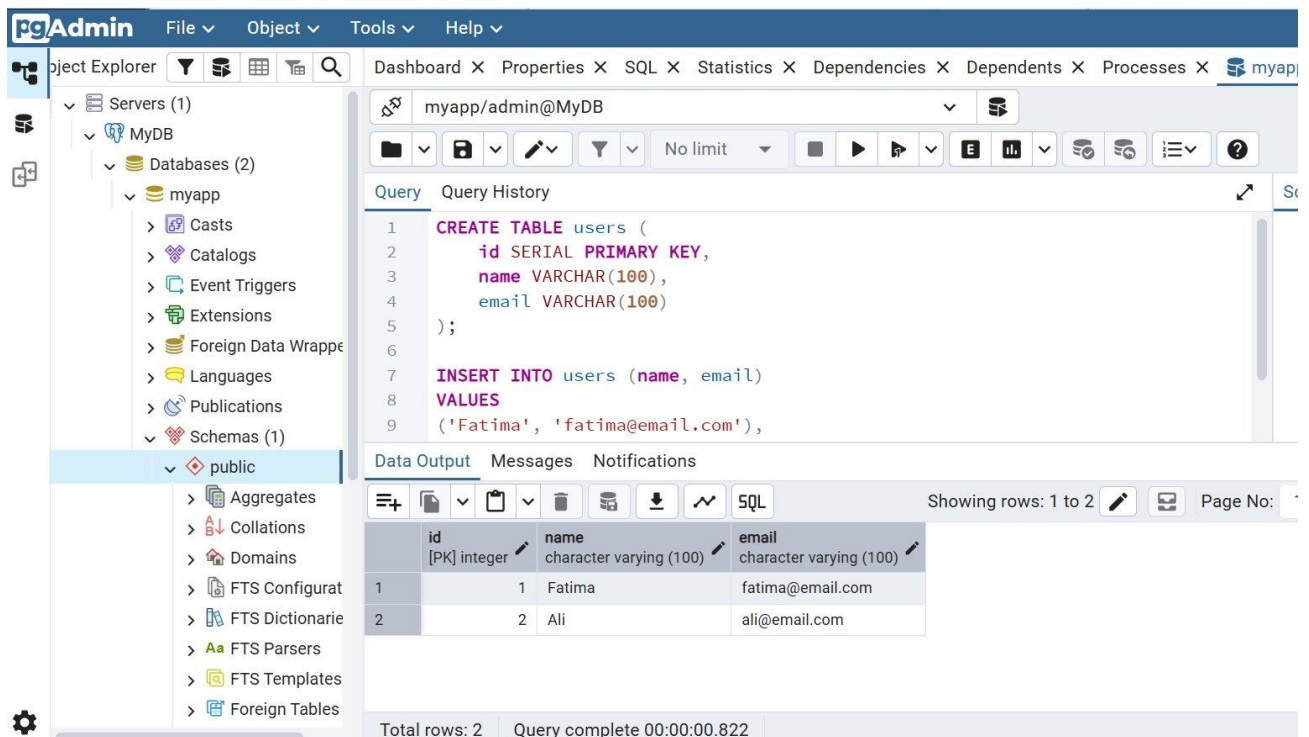


Figure 7 – Création de la table users et insertion des données (résultat : 2 lignes)

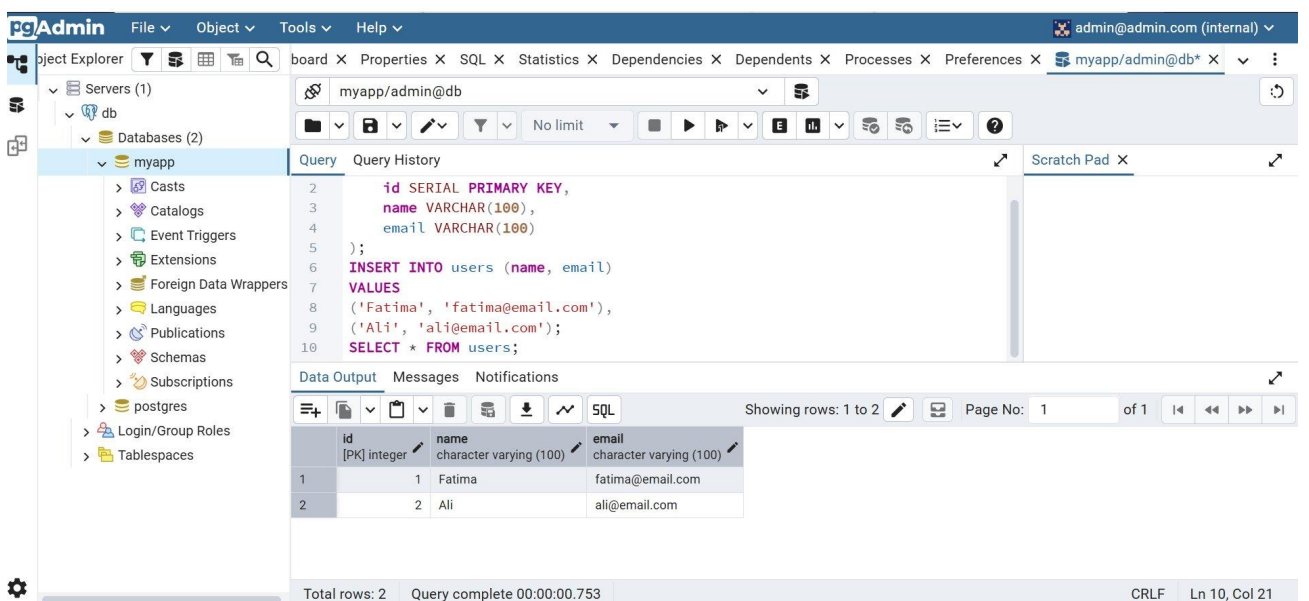


Figure 8 – Résultat SELECT * FROM users dans la base myapp (connexion db)

5. Persistance des Données

Le volume nommé db-data est monté sur /var/lib/postgresql/data, garantissant la survie des données après redémarrage des conteneurs. Pour valider ce comportement, une deuxième session d'insertion a été effectuée après redémarrage, puis un SELECT confirme la présence des 4 lignes cumulées :

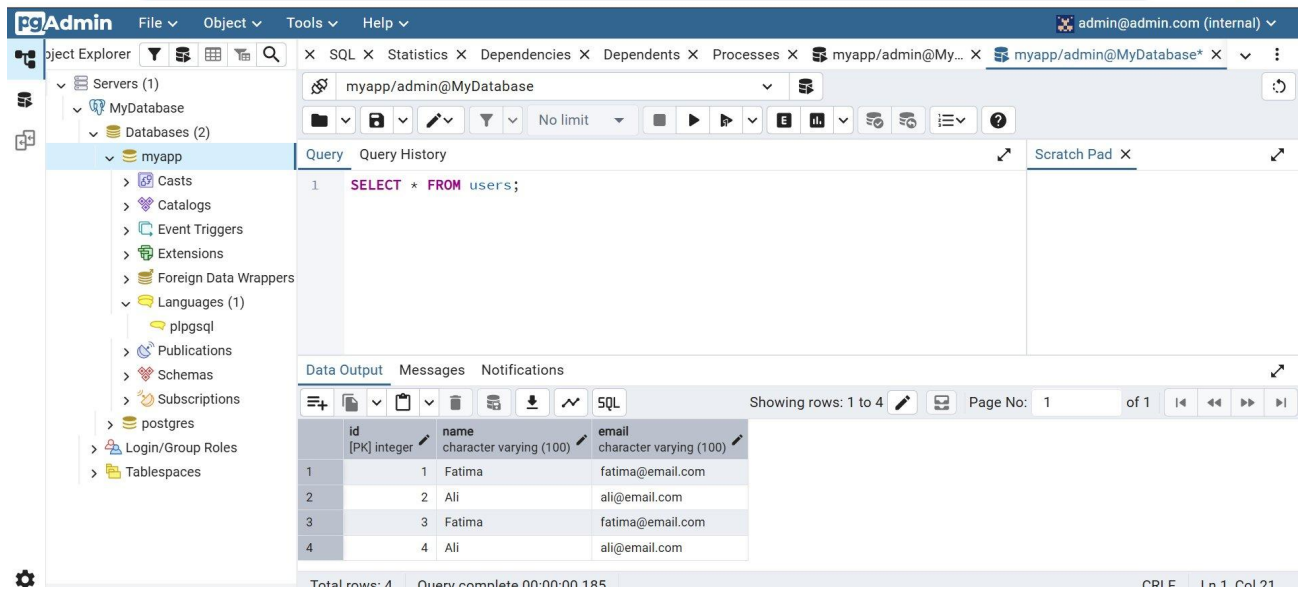


Figure 9 – Après redémarrage : 4 lignes dans la table users (données persistantes)

6. Arrêt de l'Application

La commande docker-compose down arrête et supprime les conteneurs ainsi que le réseau, tout en conservant le volume de données :

```
docker-compose down
```

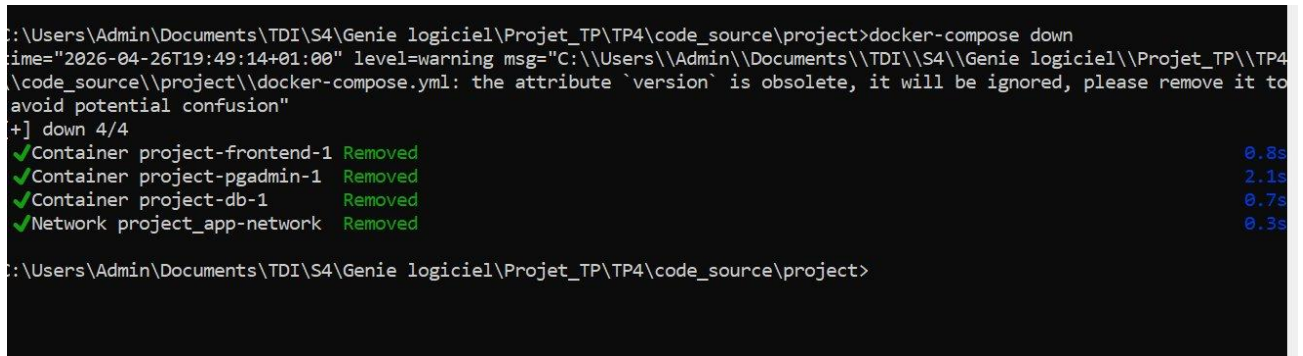


Figure 10 – Résultat de docker-compose down : 3 conteneurs et le réseau supprimés

7. Conclusion

Ce TP a permis de maîtriser l'ensemble du workflow Docker dans un contexte d'application multi-services réaliste. Les compétences acquises couvrent :

- La rédaction d'un Dockerfile optimisé pour une image Nginx légère (nginx:alpine)

- L'utilisation de `.dockerignore` pour réduire le contexte de build
- L'orchestration de plusieurs services interdépendants via Docker Compose
- La gestion des réseaux virtuels pour isoler et connecter les conteneurs
- La configuration de volumes persistants pour les données PostgreSQL
- La manipulation d'une base de données PostgreSQL via pgAdmin 4

La maîtrise de ces outils est fondamentale dans les environnements DevOps et de transformation digitale industrielle, garantissant portabilité, reproductibilité et isolation des applications.